

数据集相关

数据集

数据集生成

数据集生成策略 (self)

第一步：提取领域和场景知识

收集一些领域内的权威资料（有用信息密度肯定很小），然后从中提取出来数据信息，并将这些数据存入一个知识文件(为了提高有用信息的密度)

最好能总结出来，在某个场景下，可以进行什么动作，每个信号的合理数值范围是多少。

第二步：生成候选模板，并按照类型分类汇总

需要总结出模板的类型有哪些，然后每种类型的模板应该有很多个候选，按照每种类型的模板进行分类汇总

第三步：提炼出原子谓词

对于每个领域的每个场景，根据提炼出来的知识，生成原子谓词。

第四步：根据领域和场景选择合适的模板，生成stl公式

需要一个脚本，对每个stl公式进行语法验证--->至少可以保证stl公式是合法的。

第五步：根据 stl 公式 反 译 出自然语言

将stl公式和领域&场景&其它必要的信息 ---> llm ---> nl 描述

拷打点

(1) 如何保证stl公式与nl语义一致：

方案一：小规模数据集 ($\leq 2k$)，可以通过人工审核的方式，尽可能地保证每条数据都是语义一致的

方案二：不能保证每条数据都一定是语义一致的，这样的话，数据集规模可以做的大一些。但是只能保证语法正确，语义一致性质量只能是一般

方案三：牺牲灵活性/多样性，按照deepstl的生成模式来生产数据集（规定好映射表达方式）。

实际上这本身就存在一个矛盾点，要想保证语义一致，就无法/很难保证灵活性（也就是工程上的nl描述风格），要想保证工程化的nl描述风格，就保证不了语义一致性。只能通过人工审核来尽量保证语义一致性，但是这就限制了数据集的规模。

当前STL-DivEn和STL-Bench都是不保证语义一致性；甚至STL-DivEn数据集中语法正确都不保证，而且错误很多，语义不一致情况更多（保守估计有数千条错误数据）；STL-Bench数据集因为没有公开，不做评价，不过可以确定的是，他们也保证不了语义一致性。

(2) 如何保证数据集中数据的多样性？会不会存在很多重复的数据？

对于数据的多样性，实际上取决于模板的多样性，我们可以保证对于每个领域的每个场景，在该场景下同一个模板只进行一次填充和翻译。

(3) 如何保证翻译得到的nl是工程风格的描述？(不需要保证，没有义务)

代码块

- 1 严格来说，**不能保证**“工程表达风格”，也没有人能保证生成的nl描述是符合工程表达风格的，只能通过提示和抽样审核来**引导**模型生成类似工程需求的文本。
- 2

实际采用的方式

1. 给模型提供具体工程场景

不是只输入：G[0,120](pressure < 80)

还会告诉模型：

领域：工业控制

场景：锅炉系统

信号：pressure

因此模型更可能生成：

而不是普通口语：

这种方法本质上是利用 DeepSeek-Chat 已有的工程文本知识，通过场景条件激活相应语言风格。

2. 在 Prompt 中直接要求“工程风格”

代码块

- 1 Use professional engineering requirement language.
- 2 Use precise and unambiguous temporal expressions.

- 3 Avoid conversational or narrative wording.
- 4 Preserve all signals, thresholds and temporal scopes.

3. 使用领域术语和物理单位

工程感还来自文本内容本身，例如：

- pitch、yaw、airspeed；
- pressure、flow rate、torque；
- psi、kPa、knots、ft、rpm；
- AEB、CNC machining、battery management。

这些词天然会让文本看起来更像工程需求。但这只能说明文本具有**工程领域词汇**，不能直接证明它符合真实需求规范的写作方式。

因此不能说 **保证了工程表达风格**。更严谨的说法是：

STL-Bench/我们的数据集 生成了具有工程场景、专业信号和物理单位的自然语言，并通过 Prompt 要求其采用工程式表达；但“工程风格”主要依赖 LLM 的生成能力和少量人工抽检，没有被系统地验证或严格保证。

所以我们的做法：？

第六步：模糊化

模糊化策略：

代码块

- 1 选定模糊类型：对原子谓词进行模糊，对时间区间进行模糊，指代不确定【歧义】？
- 2
- 3 能否对于整体结构进行模糊化？----- 不好做，不想做，也没人这样做。

当前研究现状：

代码块

- 1 做模糊nl2stl方向的，论文已经中了，还没有。
- 2 在投：ClarifySTL【Tosen】
- 3 做法：
- 4 对谓词进行模糊（包括：模糊词替换 和 具体数值删除）
- 5 对时间区间进行模糊（模糊词替换）

需要保留 {模糊点&澄清}

现有数据集生成方法

deepstl

第一步：从真实文献中统计 STL 的使用规律

作者先从科研论文和部分工业文档中收集并分析了 **130 多条 STL 公式及其英文描述**，覆盖：

- 自动驾驶
- 机器人
- 电子系统
- 时间序列分析
- STL specification pattern

作者主要统计了三类信息：

1. 常见 STL 公式结构；
2. 各种 STL 算子的使用频率；
3. STL 算子在自然语言中的常见表达方式。

例如，他们统计发现：

- G 比 F 更常见；
- G、F、H、O 等一元算子比 U、S 更常见；
- 未来时算子比过去时算子更常见；
- rise、fall 和过去时算子通常出现在前置条件中；
- 未来时算子通常出现在后置条件中。

四类常见需求模板

作者把真实公式主要归为四类：

类型	典型结构	含义
Invariance / Reachability	$G_I \phi$ 、 $F_I \phi$	始终满足或最终达到
Immediate Response	$G(\phi \rightarrow \psi)$	条件发生后立即满足
Temporal Response	$G(\phi \rightarrow F_I \psi)$ 等	条件发生后一段时间内响应
Stabilization / Recurrence	$F_I G_J \phi$ 、 $G_I F_J \phi$	最终稳定或周期性发生

在收集的公式中，作者统计到：

- Invariance/Reachability：39 条；
- Immediate Response：27 条；
- Temporal Response：33 条；
- Stabilization/Recurrence：31 条。

这些统计随后被用作随机生成公式时的采样概率，而不是让所有算子均匀出现。

第二步：定义一个受限的 STL 语法

为了保证 STL 能够自动生成、自动翻译成英文，作者没有支持任意复杂 STL，而是设计了一个**三层语法结构**。

1. 底层：Simple Phrase

底层主要生成原子命题和简单布尔组合。

原子命题大致为：

代码块

```
1  x < u
2  x <= u
3  x = u
4  x >= u
5  x > u
6  not(x > u)
7  rise(x > u)
8  fall(x > u)
9  not rise(x > u)
10 not fall(x > u)
```

然后允许两个原子命题通过 `and` 或 `or` 组合：

代码块

```
1  x > 5 and y < 10
```

```
2    x <= 3 or y = 0
```

形式上为：

代码块

```
1    SP := α
2      | α and α
3      | α or α
```

因此这里最多只允许两个原子命题进行布尔组合。

2. 中层：Temporal Phrase

中层是在原子命题上增加一个时间算子。

支持的一元时间算子包括：

代码块

```
1    eventually
2    always
3    once
4    historically
```

支持的二元时间算子包括：

代码块

```
1    until
2    since
```

例如：

代码块

```
1    eventually [0:10] (x > 5)
2
3    always [10:30] (y < 2)
4
5    (x > 5) until [0:20] (y = 0)
```

形式上为：

```

1 1st PP' := UnaryTemporalOperator_I(α)
2      | α BinaryTemporalOperator_I α

```

时间区间要求满足：

代码块

```

1 0 <= lower < upper <= infinity

```

因此，从生成规则上看，时间区间通常不会出现明显的上下界颠倒。

3. 顶层：Single Nested Temporal Phrase

顶层只允许**一层时间算子嵌套**，主要是：

代码块

```

1 eventually_I always_J α
2 always_I eventually_J α

```

也就是常见的：

代码块

```

1 F G α
2 G F α

```

分别用于描述：

- 最终进入稳定状态；
- 在整个时间段内反复达到某个状态。

例如：

代码块

```

1 eventually [0:100] (
2     always [0:20] (speed < 10)
3 )

```

但是它不支持任意深度的嵌套，例如：

```
1   $\neg G(F(G(F(\dots))))$ 
```

也不普遍支持：

代码块

```
1   $F(F \varphi)$   
2   $F(\varphi1 \cup \varphi2)$   
3   $G((F \varphi1) \text{ and } (G \varphi2))$ 
```

第三步：根据四种大模板生成完整 STL

在上述三层组件之上，DeepSTL 定义完整公式：

代码块

```
1   $G\_I(SP)$   
2   $F\_I(SP)$ 
```

对应不变性和可达性。

代码块

```
1   $G(SP \rightarrow SP)$ 
```

对应立即响应。

代码块

```
1   $G(P \rightarrow TP)$ 
```

对应时间响应，其中：

代码块

```
1   $P := SP \mid TP$ 
```

最后是：

代码块

```
1 G(P -> NTP)
```

对应稳定性或重复性需求。

支持范围并不完全

对作者收集到的真实模板，该受限语法大约能够支持：

- 97.4% 的 Invariance/Reachability;
- 96.3% 的 Immediate Response;
- 42.4% 的 Temporal Response;
- 32.3% 的 Stabilization/Recurrence。

这说明它对简单、常见需求覆盖较好，但对于复杂响应和复杂嵌套覆盖明显不足。

第四步：按照经验概率随机生成 STL

DeepSTL 不是简单地均匀随机选择算子。

它将语法规则附加概率：

代码块

```
1 P(template)
2 P(operator | template)
3 P(comparison operator)
4 P(temporal operator)
5 P(Boolean operator)
```

这些概率来自前面真实 STL 文献的统计。

所以在生成数据时：

- `always` 出现得最多；
- `eventually` 次之；
- `until`、`since` 较少；
- 复杂边缘算子和过去时算子更少。

这种方式的作用是避免生成大量形式上合法、但现实中几乎没人使用的奇怪公式。

同时，生成器随机填充：

- 信号名称；
- 数值常量；
- 模式名称；
- 时间上下界；
- 比较符；
- Boolean 算子。

例如，先生成结构：

代码块

```
1  G( $\varphi_1 \rightarrow F[0,10](\varphi_2)$ )
```

再填充叶子：

代码块

```
1   $\varphi_1 := V\_In > 5$   
2   $\varphi_2 := V\_Out < 2$ 
```

得到：

代码块

```
1  G( $V\_In > 5 \rightarrow F[0,10](V\_Out < 2)$ )
```

第五步：从 STL 反向生成英文

这是 DeepSTL 质量较高的关键。

它不是让语言模型自由描述 STL，而是采用**组合式模板生成**：

代码块

```
1  STL 语法树  
2    ↓  
3  原子命题英文  
4    ↓  
5  时间短语英文  
6    ↓  
7  条件、响应关系英文  
8    ↓
```

1. 原子命题翻译

例如：

代码块

```
1  V_In > 5
```

可以生成：

代码块

```
1  the value of V_In is greater than 5
2  V_In is larger than 5
3  the signal V_In is above 5
4  the value of signal V_In exceeds 5
```

对于：

代码块

```
1  V_Out < 2
```

可以生成：

代码块

```
1  V_Out is smaller than 2
2  the value of V_Out is less than 2
3  V_Out remains below 2
```

这些候选短语不是完全均匀选择，而是根据文献中各种表达的频率进行加权。

2. 根据前置条件或后置条件调整语言

同一个原子命题处于不同位置，生成方式不同。

位于蕴含前件

例如：

代码块 > 5

处于：

代码块

```
1 G(x > 5 -> ...)
```

会使用：

代码块

```
1 if x is greater than 5
2 whenever x exceeds 5
3 when the value of x is above 5
```

位于蕴含后件

例如：

代码块

```
1 ... -> y < 2
```

会加入义务性情态动词：

代码块

```
1 y should be smaller than 2
2 y shall remain below 2
3 y must be less than 2
```

对于立即响应，还可能增加：

代码块

```
1 immediately
2 instantly
3 without any delay
4 at once
```

也就是说，生成器不仅知道公式中有哪些原子命题，还知道它们是**触发条件**还是**响应义务**。

3. 模板精炼器

论文中的 Template Refiner 主要完成：

1. 替换 subject 占位符；
2. 替换 object 占位符；
3. 随机填入信号名和数值；
4. 调整谓词形式；
5. 调整单复数和动词时态；
6. 添加副词；
7. 添加 `should`、`shall`、`must` 等情态动词。

例如原始模板可能类似：

代码块

```
1  if [subject] [predicate] [object],  
2  then [subject] should [predicate] [object]
```

填充和精炼后得到：

代码块

```
1  If the value of V_In is greater than 5,  
2  then the value of V_Out should eventually become smaller than 2 within the  
   next 10 time units.
```

最后，Assembler 负责将多个短语连接成完整句子，Randomized Sampler 从候选英文集合中随机选择指定数量的句子。

4. 时间算子的翻译

对于：

代码块

```
1   $F[0,10](\varphi)$ 
```

可能生成：

代码块

- 1 eventually within the next 10 time units φ
- 2 at some time during the following 10 time units φ
- 3 there should exist a time point within 10 time units at which φ

对于：

代码块

- 1 $G[0,10](\varphi)$

可能生成：

代码块

- 1 φ must continuously hold for the next 10 time units
- 2 φ shall remain true throughout the following 10 time units
- 3 φ should hold consecutively for 10 time units

对于：

代码块

- 1 $G(\varphi_1 \rightarrow F[0,10](\varphi_2))$

可能生成论文中的两种句子：

代码块

- 1 Globally, if the value of V_In is greater than 5,
- 2 then finally the value of V_Out should be smaller than 2 at a time point
- 3 within 10 time units.

以及：

代码块

- 1 It is always the case that when the signal V_In is larger than 5,
- 2 then eventually at sometime during the following 10 time units
- 3 the signal V_Out shall be smaller than 2.

这些英文虽然比较啰嗦，但结构和公式对应关系很明显。

STL-DivEn

STL-DivEn 的核心路线是：

人工构造少量高质量种子 → 聚类选择代表样本 → GPT-4 同时生成 NL 和 STL → 规则过滤 → 人工审核 → 合格数据反过来扩充种子集 → 迭代生成到 16,000 条。

整体流程

可以写成下面的迭代过程：

$$S_0 \xrightarrow{\text{聚类选5个示例}} \text{GPT-4生成候选集} \xrightarrow{\text{语法过滤}} \xrightarrow{\text{ROUGE去重}} \xrightarrow{\text{人工审核}} Q_0$$

然后：

$$S_1 = S_0 \cup Q_0$$

再使用扩充后的 S_1 继续选择示例、生成、过滤和审核，直到得到约 16,000 条数据。

代码块

```
1  人工种子集
2      ↓
3  Sentence-Transformers 向量化
4      ↓
5  K-means 聚类，选择代表样例
6      ↓
7  GPT-4 生成新的 NL-STL 对
8      ↓
9  STL 语法检查
10     ↓
11  ROUGE 相似度过滤
12     ↓
13  人工语义审核
14     ↓
15  加入 STL-DivEn，同时加入种子集
16     ↓
17  进入下一轮
```

第一步：人工构造120条种子数据

1. 种子集规模

一共：

- 120 条 NL-STL 对；

- 6 名领域专家；
- 3 个应用领域；
- 每个领域2名专家；
- 每个领域40条。

三个领域是：

1. 自动驾驶；
2. 机器人；
3. 电子系统。

这些种子同时覆盖：

- 基础 STL 逻辑；
- 嵌套 STL 逻辑；
- 不同应用场景；
- 不同信号、阈值和时间约束。

所谓基础逻辑，例如：

$$F_{[0,5]}(x > 3)$$

所谓嵌套逻辑，例如：

$$G_{[0,60]}(x > 3 \rightarrow F_{[0,10]}G_{[0,5]}(y < 2))$$

也就是一个时间算子的内部又包含其他时间算子。

 KGST

第二步：通过聚类选择5个代表性样例

1. 向量化

作者使用 Sentence-Transformers，将每个 NL-STL 对映射到高维向量空间。

可以理解为：

NL: Whenever the speed exceeds 50...
STL: G[0,30](speed > 50 -> ...)



被编码为一个语义向量：

$$e_i = \text{Encoder}(NL_i, STL_i)$$

不过论文没有明确说明：

- 使用的是哪一个 Sentence-Transformers 模型；
- NL 和 STL 是直接拼接还是分别编码；
- 最终选择的是距离中心最近的样本，还是直接使用聚类中心表示。

因此，论文这里只给出了总体方法，没有提供完全可复现的编码细节。

2. K-means 聚类

随后对120个向量运行 K-means：

$$K = 5$$

即把种子数据划分为5个簇，并从每个簇中选取一个代表性样本，最终得到5个示例。

这5个样例作为 GPT-4 每次生成时的 few-shot demonstrations。

3. 聚类的目的

GPT-4 很容易模仿提示词中的例子。例如提示词中多数是：

$$G(A \rightarrow FB)$$

模型生成的数据也可能大量集中在这种结构。

通过聚类，从不同簇中分别选择样例，可以尽量让提示中包含不同的：

- 时间算子；
- 布尔结构；
- 嵌套深度；
- 应用领域；
- 信号数量；
- 语言表达方式。

它主要解决的是**示例选择偏置**，而不是直接证明生成数据一定多样。

第三步：GPT-4 同时生成 NL 和 STL

第四步：STL语法过滤

【从结果上来看，他根本没有在数据集生成的过程中进行语法过滤】

【因为从所谓的半成品(很可能就是最终的成品)中，已有17k条数据，里面有大量通不过语法验证的数据】

GPT-4 生成的候选数据首先经过语法检查。

论文定义的 STL 语法主要包括：

$$\varphi ::= \alpha \mid \perp \mid \neg \varphi \mid \varphi_1 \wedge \varphi_2 \mid G_I \varphi \mid F_I \varphi \mid \varphi_1 U_I \varphi_2$$

并允许把：

- or
- ->

作为语法糖。

语法检查程序会过滤掉不满足语法规则的数据，例如：

G[0,10] x > 3 F[0,5]

括号或结构不合法。

或者：

10 <= t <= 30(G[0,5](x > 20))

把普通数学区间条件错误地当成 STL 算子。

语法过滤能够检查什么

它可以检查：

- 括号是否匹配；
- 时间算子参数是否合法；
- 操作数数量是否正确；
- 原子谓词格式是否合法；
- 整个公式能否被 STL 语法解析。

论文没有说明：

- 使用了什么 STL parser；
- 是否使用 RTAMT；

- 是否自行实现语法分析器；
- 是否检查公式时间区间和信号类型；
- 是否执行可满足性检查。

第五步：使用ROUGE过滤相似数据

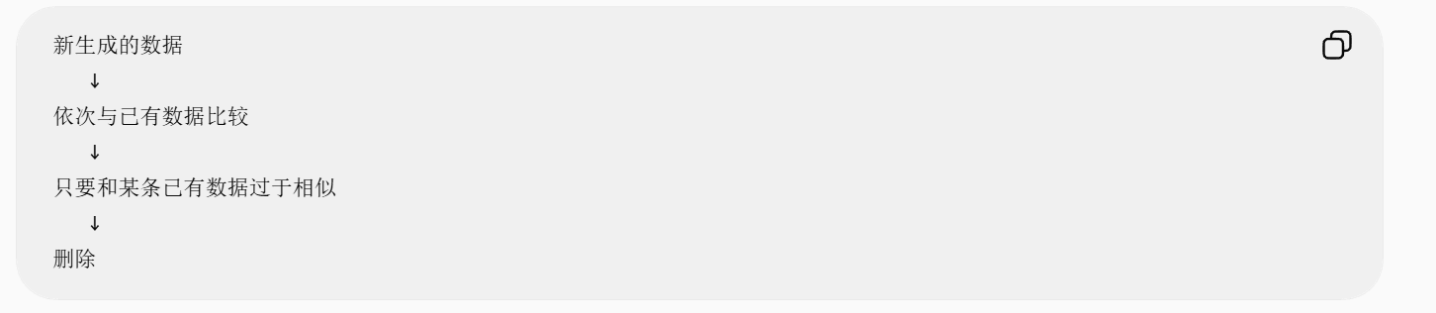
通过语法检查以后，作者计算新数据与现有种子数据之间的 ROUGE 分数。

判断规则是：

$$\text{ROUGE}(\text{generated pair}, \text{seed pair}) < 0.5$$

而且需要新数据与现有种子集中的所有数据都满足低于0.5，才被认为具有足够的多样性。

简单来说：



ROUGE过滤的实际作用

它主要过滤：

- 几乎照抄示例的数据；
- 只更换信号名称的数据；
- 只修改数值的数据；
- 大量复用相同自然语言句式的数据；
- 公式字符串高度相似的数据。

例如已有：

代码块

1 Whenever x exceeds 10, y must remain below 5.

新生成：

代码块

```
1 Whenever x exceeds 20, y must remain below 8.
```

虽然数字不同，但句式 and 结构高度相似，可能被过滤。

ROUGE过滤的局限

ROUGE 是词面重叠指标，不是语义指标。

以下两句话语义相同，但词面差异较大：

代码块

```
1 Whenever x exceeds 10, y must eventually fall below 5.  
2  
3 Each time x becomes greater than 10, it is required that  
4 at some later instant y be less than 5.
```

ROUGE 可能很低，但它们的逻辑模式实际上完全相同。

反过来，两句话词面相似，也可能只因一个否定词而语义相反：

代码块

```
1 y must remain below 5  
2 y must not remain below 5
```

因此：

ROUGE < 0.5 只能提高词汇和表述多样性，不能保证逻辑结构或语义模式真正多样。

论文也没有明确说明使用的是：

- ROUGE-1;
- ROUGE-2;
- ROUGE-L;
- 还是多个 ROUGE 分数的组合。

也没有明确说明 ROUGE 是只计算 NL、只计算 STL，还是计算拼接后的 NL-STL 对

第六步：人工验证 NL-STL 语义一致性

通过语法和相似度过滤的数据，还需要人工审核。

论文给出的信息是：

- 7名标注人员；
- 标注人员接受过 STL 使用和表达方面的培训；
- 人工标注持续2个月；
- 重点验证自然语言与 STL 是否一致。

第七步：将合格数据加入种子集并继续迭代

STL-Bench

核心思想：**先生成确定的 STL，再让大模型写自然语言**

STL-Bench 不是让大模型先写需求，再反向标注 STL；而是采用：

场景与模板 -> 参数实例化 -> 确定STL标签 -> 生成自然语言

也就是说，**STL 公式是先由程序化模板确定的，LLM 只负责把已经确定的公式“语言化”**。

这是整个数据集构造方法中最关键的设计，因为它能够减少：

- LLM 自由生成 STL 导致的公式错误；
- 自然语言和 STL 之间的语义漂移；
- 人工从自然语言反向标注公式时的主观性。

论文将这种方法称为 **template-anchored、verification-driven pipeline**，即“模板锚定、验证驱动”的数据生成流程

第一步：建立工程领域和场景体系

作者首先确定数据集要覆盖的真实 CPS 工程领域，而不是使用 `x1、x2、x3` 这类抽象信号。

STL-Bench 覆盖六个领域：

领域	示例场景
自动驾驶	AEB、ACC、车道保持、自动泊车、牵引力控制
机器人	抓取、焊接、碰撞避免、装配、移动导航
工业控制	反应器、数控加工、传送带、液压机、锅炉
环境监测	室内环境、水质、空气质量、温室、噪声
电气系统	电池管理、电机驱动、电网、太阳能板
航空航天	高度保持、起降、无人机巡检、卫星姿态

最终形成：

- 6 个工程领域；
- 33 个具体场景；
- 41 个规范化信号变量。

例如：

代码块

```
1 altitude
2 speed
3 acceleration
4 pressure
5 temperature
6 torque
7 voltage
8 current
9 humidity
10 pitch
11 roll
12 yaw
```

这样生成的公式不再是：

代码块

```
1 G[0,10](x1 > 5)
```

而更接近真实工程需求：

代码块

```
1 G[0,10](pressure > 5)
```

这一阶段主要决定了数据集中的**领域语义空间和信号词表**。

第二步：设计参数化 STL 模板

针对每一个场景，作者设计一组可参数化 STL 模板。

每个模板会规定：

- 1. 可以使用哪些 STL 算子；
- 2. 可以使用哪些信号；
- 3. 信号数值的合理范围；
- 4. 时间区间的合理范围；
- 5. 阈值和单位；
- 6. 是否需要中间计算；
- 7. 公式的嵌套复杂度。

数据集支持的主要算子包括：

代码块

- 1 Globally
- 2 Finally
- 3 Until
- 4 Since
- 5 Historically
- 6 Once
- 7 Rise
- 8 Fall
- 9 and
- 10 or
- 11 Not
- 12 imply

例如，一个模板可能是：

$$G_{[t_1,t_2]} \left(p_1 \rightarrow F_{[0,d]} p_2 \right)$$

其中：

- p_1 ：触发条件；
- p_2 ：响应条件；
- t_1, t_2 ：外层观察窗口；
- d ：响应期限；
- 信号和阈值由场景决定。

在自动驾驶场景中，可以实例化为：

$$G_{[0,120]} \left(distance < 20 \rightarrow F_{[0,5]} (speed < 10) \right)$$

模板不仅约束公式结构，还约束数值的物理合理性。例如，汽车速度、飞行高度、液压压力会分别使用不同的范围，而不是完全随机采样。



第三步：按照 1—6 级控制公式复杂度

没有把所有公式混在一起，而是将样本分为六个复杂度等级。

等级	主要特征	采样比例
1	单一算子、一个原子谓词	25%
2	两个算子、2—3 个谓词	25%
3	三个算子、3—5 个谓词	20%
4	多阶段时序约束、4—6 个谓词	15%
5	深层嵌套、多分支、5—7 个谓词	10%
6	高度复杂、6—8 个谓词、链式计算	5%

复杂度主要由以下因素共同决定：

- STL 树的深度；
- 时序算子数量；
- 布尔算子数量；
- 原子谓词数量；
- 工具调用次数；
- 中间计算链长度。

例如：

Level 1

$$G_{[0,30]}(temperature < 80)$$

Level 3

$$G_{[0,120]}(pressure > 50 \rightarrow F_{[0,10]}(flow_rate > 20))$$

Level 5—6

可能出现：

$$G_{[0,120]}[temperature < 80 \wedge (pressure > 50 \rightarrow F_{[0,15]}(flow_rate > 20 \wedge H_{[0,5]}(voltage > 200)))]$$

因此，STL-Bench 并不是单纯随机扩展样本数量，而是显式控制公式复杂度分布。

 ReasonSTL

第四步：实例化模板，先确定 STL 标准答案

选定模板后，程序会采样：

- 场景；
- 信号变量；
- 比较关系；
- 时间区间；
- 数值阈值；
- 物理单位；
- 工具调用要求。

然后直接生成目标 STL JSON 树。

论文没有把 STL 保存为普通字符串，而是保存为结构化 JSON，例如：

代码块

```
1  {
2    "STL": {
3      "Operation": "Finally",
4      "Time": [0, 1800],
5      "Leftaction": null,
6      "Rightaction": {
7        "Operation": "and",
8        "SubQueries": [
9          "altitude>3048.0",
10         "speed>=102.89"
11       ]
12     }
13   }
```

```
13     }  
14 }
```

其含义是：

$$F_{[0,1800]} (altitude > 3048.0 \wedge speed \geq 102.89)$$

这种 JSON 树有几个作用：

- 明确算子层级；
- 避免括号和优先级歧义；
- 便于检查算子参数数量；
- 便于递归比较预测公式与标准公式；
- 便于强化学习时给予局部奖励。

因此，数据集的标准答案本质上是一个**结构化 STL AST**。

 ReasonSTL

第五步：生成自然语言描述

使用 DeepSeek-Chat 进行受控语言实现

在 STL JSON 和参数全部确定后，作者调用 DeepSeek-Chat，并给定一些必要的信息，将固定公式改写为工程风格的自然语言。【中英文独立生成，避免因为中英文表达风格不同导致质量降低】

（在生成自然语言的时候，还引入了一些单位使用说明，指导模型按照什么单位进行翻译，后续做 nl2stl 的时候，需要使用确定性工具进行单位转换）

对于 rise / fall 事件语义还需要做特殊处理

第六步：进行自动验证

生成候选样本后，不会直接加入数据集，而是经过规则验证器。

验证内容包括：

1. 自然语言检查

- 长度是否合理；
- 是否为空；
- 是否包含异常内容；
- 是否符合语言要求。

2. JSON 格式检查

- JSON 是否可解析；

- 必需字段是否存在；
- 字段类型是否正确。

3. STL 树结构检查

- 算子是否合法；
- 一元算子是否只有一个子公式；
- 二元算子是否有左右子公式；
- 布尔算子子节点是否正确；
- 时间算子是否有合法区间。

4. 时间区间检查

要求：

$$0 \leq a \leq b$$

避免出现：

[20, 10]

[-5, 10]

等无效区间。

5. 谓词检查

检查：

- 信号名称是否合法；
- 比较运算符是否合法；
- 阈值是否为数值；
- 一个原子谓词中是否错误包含多个逻辑条件。

例如错误叶节点：

代码块

1 pressure>10 and temperature<80

可以确定性地修复成：

代码块

1 {
2 "Operation": "and",

```
3     "SubQueries": [  
4         "pressure>10",  
5         "temperature<80"  
6     ]  
7 }
```

6. 工具调用检查

- 工具名称是否正确；
- 参数是否符合接口；
- 调用是否可执行；
- 单位转换结果是否正确；
- 算术结果是否正确；
- 工具输出是否确实写入最终 STL。

例如：

代码块

```
1  30 minutes → 1800
```

但最终 JSON 写成 `[0, 180]`，该样本会被判定为不一致。

7. 只允许确定性修复

作者规定：

- 能通过固定规则唯一修复的，进行修复；
- 需要主观判断的，直接删除。

例如：

- JSON 缺少一个确定可补充的字段：可以修复；
- 单个字符串中混入 `and`：可以转换为布尔节点；
- 工具调用无法执行：删除；
- 自然语言与 STL 是否语义一致存在主观争议：不会进行自由式自动修改。

这一原则可以避免验证程序在修复过程中重新引入不可靠标签。

第七步：对生成的数据进行去重

作者进行了两阶段去重。

第一阶段：符号级去重

检查：

- 规范化后的自然语言文本；
- STL 树结构哈希；
- 谓词和参数赋值。

例如下面两条虽然空格不同：

代码块

```
1  G[0,10](speed>20)
2  G [0, 10] ( speed > 20 )
```

规范化后会被视为相同样本。

同一 STL 模板、同一信号、同一阈值、同一时间参数的重复实例也会被过滤。

第二阶段：语义近重复去重

作者使用：

代码块

```
1  Qwen3-Embedding-4B
```

对自然语言进行向量编码。

当两个样本余弦相似度超过：

代码块

```
1  0.9
```

就会被聚到同一近重复簇中。

每个簇优先保留：

- STL 复杂度更高的样本；
- 领域语义更清楚的样本；
- 自然语言质量更好的样本。

因此，它不仅去除完全相同的数据，也尝试去除大量“只换几个词”的近重复表达

八、人工审核

自动验证之后，作者又抽取分层样本进行人工审计。

抽样覆盖：

- 英文和中文；
- 六个工程领域；
- 不同复杂度；
- 不同工具调用类型；
- 状态谓词和边缘事件。

人工检查内容包括：

- NL 与 STL 是否语义一致；
- JSON 是否正确；
- 工具计算结果是否正确；
- 中英文专业术语是否合理；
- 公式是否可读；
- `Rise/Fall` 是否使用正确。

但是需要特别注意：

作者没有逐条人工检查全部 28,880 条数据，只进行了分层抽样审核。

论文明确承认，STL-Bench 仍然可能存在：

- 语义歧义；
- 标注噪声；
- 模板实例化错误；
- 自然语言和 STL 不完全对齐；
- 错误的参考公式。

测试集加入人工编写测试集

作者另外构造了大约 200 条人工需求：

- 不使用原有数据生成模板；
- 人工分别撰写英文和中文需求；

- 人工构造对应 STL JSON；
- 再经过相同验证流程。

这个测试集主要用于判断模型能否泛化到模板之外的自然语言。

Tips

SP/TP/NTP/P解释

符号	全称	含义
SP	Simple Phrase	简单短语，不包含时间算子
TP	Temporal Phrase	时间短语，包含一层时间算子
NTP	Nested Temporal Phrase	单层嵌套时间短语
P	辅助语法变量	SP 或 TP，不是独立层级

SP

1. SP: Simple Phrase

由原子命题或最多两个原子命题的布尔组合构成：

$$SP ::= \alpha \mid \alpha \wedge \alpha \mid \alpha \vee \alpha$$

其中原子命题 α 可以是：

```
sig1 > 5  
not(sig1 > 5)  
rise(sig1 > 5)  
fall(sig1 > 5)
```



例如：

```
sig1 > 5  
sig1 > 5 and sig2 <= 10  
rise(sig1 > 5) or sig2 == 0
```



特点：没有 `always`、`eventually`、`until` 等时间算子。



TP

2. TP: Temporal Phrase

在原子命题上施加一个时间算子。

其核心形式为：

$$TP' ::= UTO_I(\alpha) \mid \alpha BTO_I \alpha$$

其中：

- 一元时间算子 `UTO`：F、G、O、H
- 二元时间算子 `BTO`：U、S

例如：

```
eventually [0:10] (sig1 > 5)
always [0:20] (sig2 < 3)
(sig1 > 5) until [0:10] (sig2 == 0)
historically [0:5] (sig3 <= 8)
```



DeepSTL 还允许对整个 `TP'` 加：

```
not
rise
fall
not rise
not fall
```



例如：

```
rise(eventually [0:10] (sig1 > 5))
```



特点：包含时间算子，但时间算子内部通常只允许原子命题，而不是复杂 SP。



NTP

3. NTP: Single Nested Temporal Phrase

表示恰好一层时间算子嵌套，DeepSTL 只支持两种：

$$NTP ::= F_I G_J(\alpha) \mid G_I F_J(\alpha)$$

例如：

```
eventually [0:100] (  
  always [0:20] (sig1 < 5)  
)
```



含义：未来 100 个时间单位内，存在某个时刻，从该时刻开始 `sig1 < 5` 连续保持 20 个时间单位。

以及：

```
always [0:100] (  
  eventually [0:20] (sig1 < 5)  
)
```



含义：在前 100 个时间单位中的每个时刻，之后 20 个时间单位内都必须出现一次 `sig1 < 5`。

DeepSTL 不支持任意嵌套，如：

```
always(eventually(always(...)))  
eventually(eventually(...))
```



4. P：辅助语法变量

论文定义：

$$P ::= SP \mid TP$$

所以 `P` 不是新结构，只表示前置条件可以是：

- 一个简单条件 `SP`
- 或一个带时间约束的条件 `TP`

例如在：

$$G(P \rightarrow TP)$$

中，`P` 可以是：

```
sig1 > 5
```



也可以是：

```
always [0:10] (sig1 > 5)
```



完整示例：

```
always (  
  always [0:10] (sig1 > 5)  
  ->  
  eventually [0:20] (sig2 < 3)  
)
```



聚类选择样本

代码块

- 1 先把120条人工种子按照“相似程度”自动分成若干组，再从每组挑一条最能代表该组的数据，作为GPT-4 的示例。